
Chapter 0 | 欢迎：这门课会带你去哪

- 我们要从一个只有 0 和 1 的世界，搭出一个能认数字的 NPU。
- 路上会用到：二进制、内存、CPU、矩阵、神经网络、卷积。
- 这一章先把这些词变成你熟悉的东西。

Chapter 0 | 电脑里只有 0 和 1

所有电子设备内部只有两种状态：有电/没电，通常写成 1 和 0。图片、声音、文字，最终都变成一串 0 和 1。

一个 0 或 1 叫一个 bit（比特）；8 个 bit 叫一个 byte（字节）。int8 的意思就是“用 8 个 bit 表示一个整数”。

- bit = 一位二进制，只能是 0 或 1。
- byte = 8 个 bit。
- int8 = 8 位有符号整数，范围 -128 到 127。

图 A：电脑里只有 0 和 1，每一位代表一个“位权”。

二进制 10110 = ?



$$= 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 22$$

Chapter 0 | 二进制怎么算

十进制每一位代表 10 的几次方：个位、十位、百位。二进制每一位代表 2 的几次方：1、2、4、8、16...

比如 $10110 = 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 22$ 。反过来， $22 = 16 + 4 + 2 = 10110$ 。

- 从右往左数：第 0 位是 1，第 1 位是 2，第 2 位是 4...
- 把位上是 1 的位权加起来就是十进制。
- 后面所有“int8 权重”都按这个规则理解。

图 A：电脑里只有 0 和 1，每一位代表一个“位权”。

二进制 $10110 = ?$



$$= 1 \times 16 + 0 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 = 22$$

Chapter 0 | 内存：带编号的格子

把内存想象成一排带门牌号的储物柜。地址（address）是门牌号，数据（data）是柜子里放的东西。

CPU 说“读地址 2”，内存就把 2 号柜子里的值还给它；CPU 说“写地址 3，内容 7”，内存就把 3 号柜子改成 7。

- 地址 = 位置编号。
- 数据 = 存在这个位置的值。
- Hack 有 32K 个 16 位格子，共 64KB。

图 F：地址像门牌号，数据像屋里的人。



Chapter 0 | CPU：听指令做事的工人

CPU 是一个“只会按说明书干活”的工人。说明书就是程序，程序里每一条都是指令，比如“把 A 的值加 1”。

CPU 的工作循环是：取指令 -> 看懂它 -> 执行它 -> 取下一条。这个循环叫“取指-译码-执行”。

- 程序 = 指令的列表。
- 指令 = 让 CPU 做一件小事。
- Hack 的 ALU 只会加减法和按位运算。

Chapter 0 | 乘法为什么贵

3×4 可以看成 $4+4+4$ 。硬件里乘法确实可以用“移位+加法”实现，但每一位都要一堆门电路，位数越多越贵。

CNN 要做几万到几亿次乘法。如果 CPU 一次只能做一个，就会慢得离谱。NPU 的思路是：把很多乘法器同时铺开。

- 乘法 = 重复加法 / 移位相加。
- 一个乘法器不贵，几万个乘法器一起干活才快。
- 这就是“并行”的朴素含义。

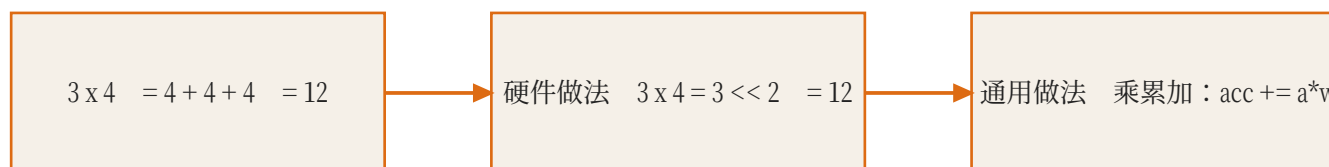


图 B：乘法看起来简单，但硬件里需要很多门电路；
所以 NPU 把乘法器铺成阵列并行做。

Chapter 0 | 矩阵：数字表格

矩阵就是一张数字表格。比如 2×2 矩阵有 2 行 2 列。一张 28×28 的灰度图片，也可以看成 28 行 28 列的数字表格。

神经网络里的权重通常也组织成矩阵，所以“算神经网络”很大程度就是“算矩阵”。

- 矩阵 = 有行有列的数字表格。
- 图片 = 数字表格。
- 权重 = 数字表格。
- 卷积/全连接最终都变成矩阵运算。

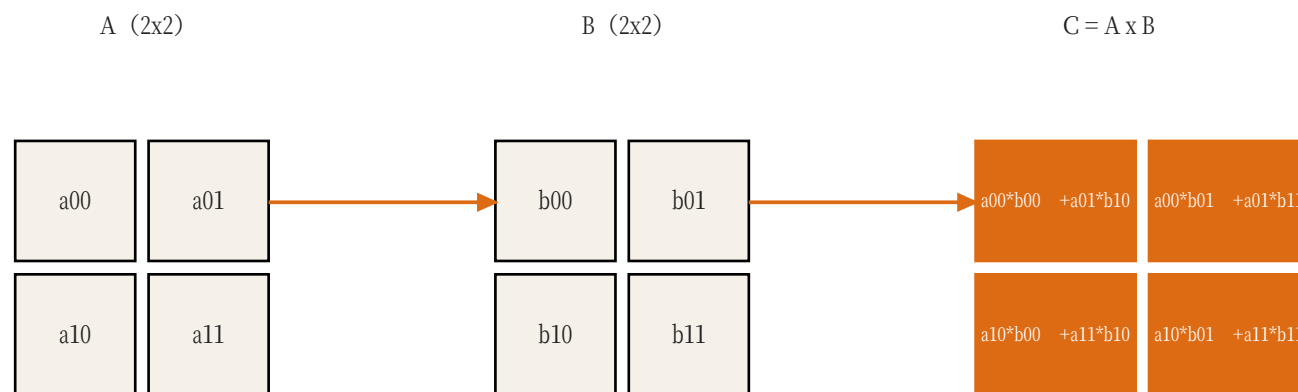


图 C : C 的每个格子 = A 的一行 与 B 的一列 逐项相乘再相加。

Chapter 0 | 矩阵乘法

$C = A \times B$ 时， C 的第 i 行第 j 列 = A 的第 i 行 与 B 的第 j 列 逐项相乘再相加。

比如 $C[0][0] = a_{00} \times b_{00} + a_{01} \times b_{10}$ 。每一个这样的“乘加”就是一个 MAC。矩阵乘法的计算量 = 所有 MAC 的数量。

- MAC = 一次乘 + 一次加。
- 矩阵乘法 = 一堆 MAC。
- NPU 的脉动阵列就是让这些 MAC 并行发生。

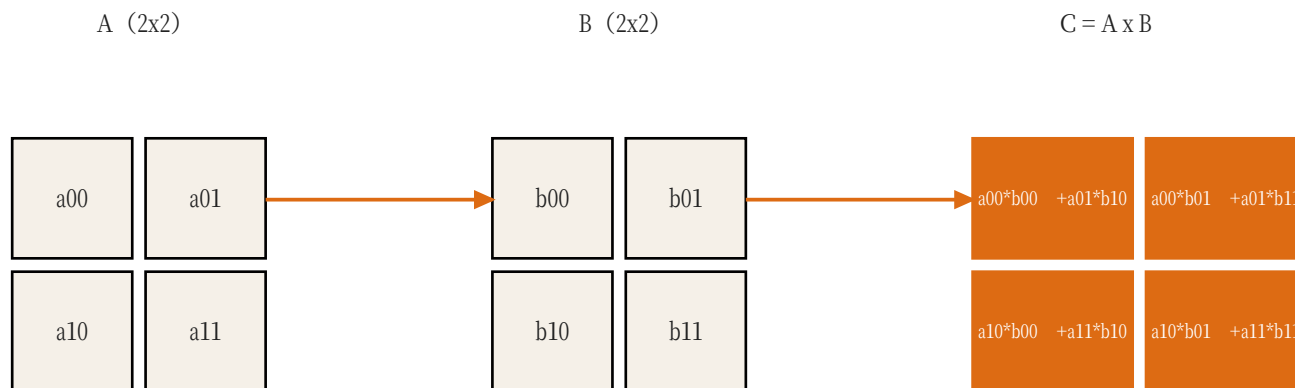


图 C：C 的每个格子 = A 的一行 与 B 的一列 逐项相乘再相加。

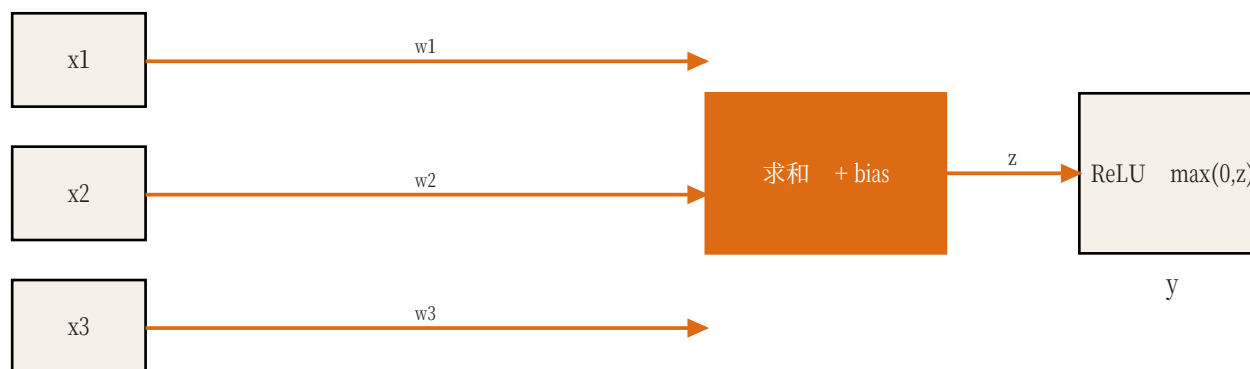
Chapter 0 | 神经网络：加权求和

一个神经元做的事：把每个输入 x 乘上权重 w ，全部加起来，再加一个偏置 b ，最后过激活函数（比如 ReLU：负数变 0）。

很多个神经元叠在一起，就组成神经网络。训练就是不断调整这些权重，让输出接近正确答案。

- $z = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$ 。
- ReLU：如果 z 小于 0，就输出 0。
- 推理 = 用已经训练好的权重做这些计算。

图 D：神经元把输入分别乘以权重、加总、再加偏置，最后过一个激活函数。



Chapter 0 | 卷积：滑动窗口

卷积就是“用一个小窗口（卷积核）在图片上滑动，每到一个位置做一次加权求和”。

5x5 图片 + 3x3 卷积核、stride=1、无 padding，输出是 3x3。公式：输出边长 = 输入边长 - 核边长 + 1。

- 卷积核 = 小窗口 + 一组权重。
- 滑动一步叫 stride。
- 卷积能提取图片里的局部特征。

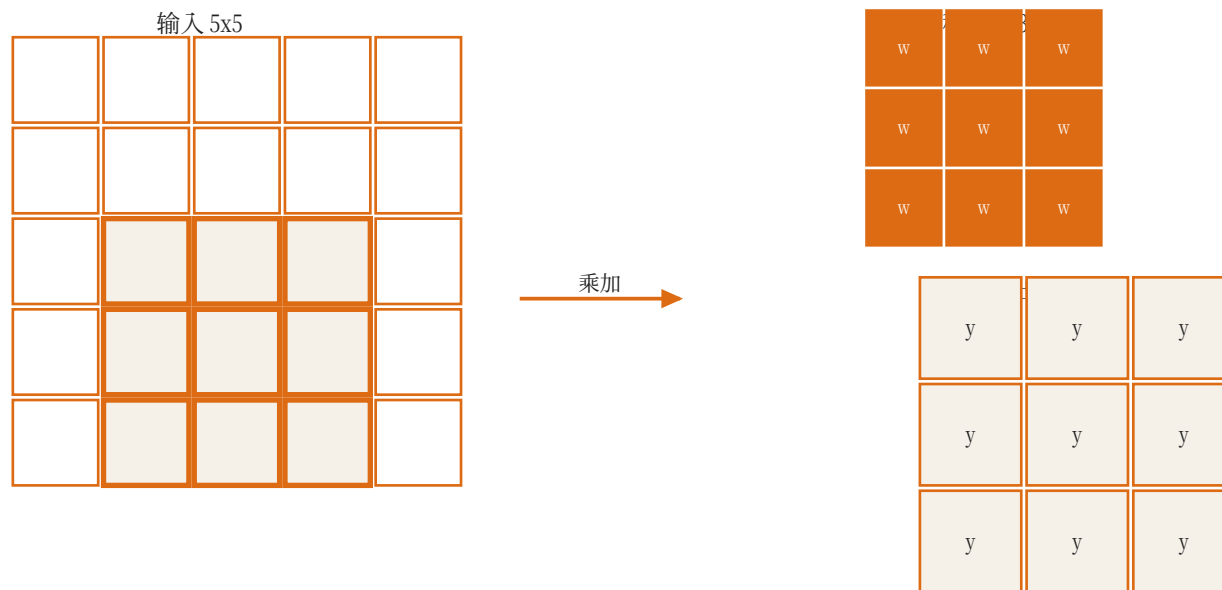


图 E：卷积核像手电筒一样在图片上滑动，每滑到一个位置就做一次加权求和。

Chapter 0 | 为什么要硬件加速

假设一层卷积要做 10 万次 MAC。CPU 一次做一个，需要 10 万步；如果有 $8 \times 8 = 64$ 个乘法器同时做，理想情况下只要约 1563 步。

硬件加速的本质就是：把“一个乘法器重复用很多次”改成“很多乘法器同时用”。

- 串行：一次一个 MAC。
- 并行：64 个 MAC 同时发生。
- NPU = 为并行 MAC 设计的专用硬件。

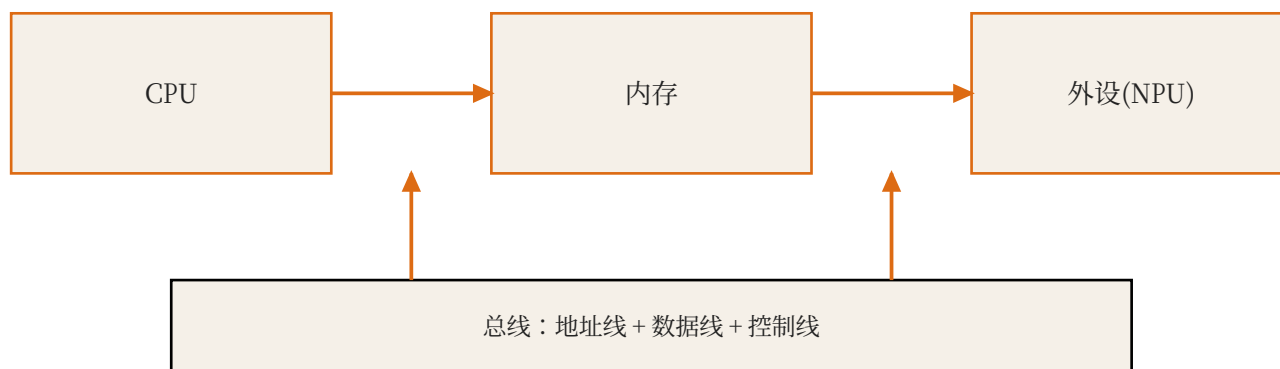
Chapter 0 | 总线：把设备连起来

CPU、内存、外设都挂在同一条“马路”上，这条马路叫总线。总线里有地址线（说找谁）、数据线（传内容）、控制线（说读写）。

MMIO 就是让外设也在这条马路上有一个门牌号。

- 总线 = 地址线 + 数据线 + 控制线。
- 每个设备有唯一地址段。
- CPU 读写外设和读写内存用的是同一种指令。

图 G：所有设备都挂在同一条“马路”上，用地址区分找谁。



Chapter 0 | MMIO：把外设当内存

Memory-Mapped I/O 的意思是：外设的寄存器也有地址。CPU 往“0x7000”这个地址写 1，NPU 就知道“开始干活”。

对 CPU 来说，控制 NPU 和控制 RAM 没区别：都是读地址、写地址。对 NPU 来说，它只是发现“有人在我的地址上读写”。

- 寄存器 = 外设身上的小格子。
- MMIO = 给这些小格子发门牌号。
- CPU 写命令、轮询状态，都是普通内存指令。

Chapter 0 | NPU 是什么

NPU (Neural Processing Unit) 是专门算神经网络的外设。它内部有：命令/状态寄存器、存放权重和数据的 SRAM、一排排乘法器（脉动阵列）、控制这些部件的小状态机。

CPU 负责“指挥”，NPU 负责“埋头算”。

- NPU = 神经网络加速器。
- CPU 调度，NPU 计算。
- 本课程要从零把 NPU 搭出来。

Chapter 0 | 本课程地图

- Ch1：定系统架构和 MMIO 协议。
- Ch2：做 PE 和脉动阵列。
- Ch3：做控制器和卷积数据通路。
- Ch4：训练并量化一个小模型。
- Ch5：挂进 Hack 整机，端到端推理。

Chapter 0 | 你需要掌握的词汇

- bit / byte / int8。
- 地址 / 数据 / 寄存器。
- 指令 / 程序 / CPU。
- MAC / 矩阵 / 卷积。
- 总线 / MMIO / NPU。

Chapter 0 | 本章作业

- 完成 Assignment.pdf 的 10 道题。
- 用手算完成 2×2 矩阵乘法。
- 用自己的话写 5 个术语解释。
- 向朋友解释一次“MMIO 是什么”。

Chapter 0 | 总结

- 电脑只有 0 和 1，但可以表达一切。
- 内存是格子，地址是门牌号。
- 神经网络 = 大量加权求和。
- NPU 把 MAC 并行化，CPU 只负责指挥。
- 下一章开始设计系统架构。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k 。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。